Project Executable Files

Date	16 October 2026
Team ID	
Project Name	Personalized Travel Itinerary Planner
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes (backend/database/schema.sql)
5	Environment configuration file (.env.example or similar)	Yes (backend/.env.example)
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA (Web Application)
8	Dockerfile / Containerization config (if applicable)	Yes (render.yaml)
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
PERSONALIZED TRAVEL ITINERARY PLANNER/
├── backend/
│   ├── app/
│   │   ├── routes/            # preferences, recommendations, itineraries, trips, export
│   │   ├── services/          # gemini_service, vibe_engine, route_optimizer (2-opt TSP)
│   │   ├── models.py          # Pydantic data schemas
│   │   └── supabase_client.py  # Supabase DB integration
│   ├── database/schema.sql     # PostgreSQL DDL & Seed Destinations
│   ├── requirements.txt        # FastAPI dependencies
│   ├── .env.example            # Environment template
│   └── main.py                # FastAPI server entrypoint
├── frontend/
│   ├── src/
│   │   ├── components/        # PreferenceForm, ItineraryTimeline, RouteOptimizer, etc.
│   │   ├── services/          # API integration & Supabase Client
│   │   ├── App.jsx            # Main UI Shell
│   │   └── index.css          # Responsive Glassmorphism Styling
│   ├── package.json           # React dependencies
│   └── vite.config.js
├── render.yaml                # Render deployment configuration
└── README.md                  # Complete setup instructions
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	
Login Credentials (Demo)	[Email: demo@travelplanner.com Password: Demo@12345 (or Guest Access enabled)
Platform / Hosting Provider	Vercel (Frontend SPA) & Render (FastAPI Backend) & Supabase (PostgreSQL)
Repository Link	
Demo Video Link	

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:
- Node.js (v18+) & Python (v3.10+)
- Git installed

Step 1: Clone repository
```
git clone https://github.com/[your-username]/Personalized-Travel-Itinerary-Planner.git
cd "Personalized Travel Itinerary Planner"
```

Step 2: Start Backend (FastAPI)
```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
(API runs at http://localhost:8000)
```

Step 3: Start Frontend (React + Vite)
```
cd ../frontend
npm install
npm run dev
(Web App runs at http://localhost:5173)
```

Step 4: Open Browser
Navigate to http://localhost:5173 to create and optimize travel itineraries.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Render Free Tier initial spin-up cold start (30–45s delay on first API call).	Built-in frontend loading spinner with automatic retry on wakeup.
2	Gemini AI API rate limits on free-tier API keys.	System features deterministic fallback mock itineraries for uninterrupted local testing.
3	Offline mode cannot sync new trips to cloud Supabase.	Automatic browser LocalStorage fallback saves trips offline until reconnection.